

Top Coding Interview Questions

(0-3 years)

5 - 7 LPA

Swipe>>

Follow to Get this

@codebuilderhq

1️⃣ Move All Hashes to Front

📌 Problem Statement:

Given a string, move all # characters to the beginning while preserving the order of the remaining characters.

- 🔷 **Input:** "a#b#c#d"
- 🔷 **Output:** "###abcd"

💻 Java Code:

```
class Main {
    public static void main(String[] args) {
        String s = "a#b#c#d";
        StringBuilder hash = new StringBuilder();
        StringBuilder normal = new StringBuilder();

        for(char ch : s.toCharArray()) {
            if(ch == '#')
                hash.append(ch);
            else
                normal.append(ch);
        }

        System.out.println(hash.toString() + normal.toString());
    }
}
```

⏱ Time Complexity: **O(n)**

📦 Space Complexity: **O(n)**

@codebuilderhq

2️⃣ String Compression

📌 Problem Statement:

Compress a string by replacing sequences of repeating characters with the character followed by its count.

- 🔷 **Input:** "aaabbcccdd"
- 🔷 **Output:** "a3b2c3d2"

💻 Java Code:

```
class Main {
    public static void main(String[] args) {
        String s = "aaabbcccdd";
        StringBuilder result = new StringBuilder();
        int count = 1;

        for(int i = 1; i < s.length(); i++){
            if(s.charAt(i) == s.charAt(i-1)){
                count++;
            } else {
                result.append(s.charAt(i-1)).append(count);
                count = 1;
            }
        }

        result.append(s.charAt(s.length()-1)).append(count);
        System.out.println(result);
    }
}
```

⏱️ Time Complexity: **O(n)**

📦 Space Complexity: **O(n)**

@codebuilderhq

3️⃣ Remove Duplicates from Sorted Array

📌 Problem Statement:

Given a sorted array, remove duplicates in-place and return the new length of the unique list.

- 🔷 **Input:** [2, 2, 3, 3, 3, 4, 5, 5]
- 🔷 **Output:** [2, 3, 4, 5]

💻 Java Code:

```
class Main {
    public static void main(String[] args) {
        int[] nums = {2,2,3,3,3,4,5,5};
        int index = 1;

        for(int i = 1; i < nums.length; i++){
            if(nums[i] != nums[i-1]){
                nums[index++] = nums[i];
            }
        }

        System.out.println("New Length: " + index);
        System.out.print("Modified Array: ");
        for(int i = 0; i < index; i++){
            System.out.print(nums[i] + " ");
        }
    }
}
```

⏱ Time Complexity: **O(n)**

📦 Space Complexity: **O(1)**

@codebuilderhq

4️⃣ First Non-Repeating Character

📌 Problem Statement:

Find the first character in a string that does not repeat. If no such character exists, return -1.

- 🔷 **Input:** "swiss"
- 🔷 **Output:** "w"

💻 Java Code:

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        String s = "swiss";
        Map<Character, Integer> map = new LinkedHashMap<>();

        for(char ch : s.toCharArray()) {
            map.put(ch, map.getOrDefault(ch, 0) + 1);
        }

        for(char ch : map.keySet()) {
            if(map.get(ch) == 1) {
                System.out.println(ch);
                return;
            }
        }

        System.out.println(-1);
    }
}
```

⏱️ Time Complexity: **O(n)**

📦 Space Complexity: **O(n)**

@codebuilderhq

5️⃣ Rotate Array by K Positions

📌 Problem Statement:

Rotate an array to the right by k positions.

- 🔷 **Input:** Array: [1, 2, 3, 4, 5] k = 2
- 🔷 **Output:** [4, 5, 1, 2, 3]

💻 Java Code:

```
class Main {
    public static void main(String[] args) {
        int[] nums = {1,2,3,4,5};
        int k = 2;
        k = k % nums.length;
        reverse(nums, 0, nums.length - 1);
        reverse(nums, 0, k - 1);
        reverse(nums, k, nums.length - 1);

        for(int n : nums) System.out.print(n + " ");
    }
    static void reverse(int[] arr, int start, int end){
        while(start < end){
            int temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;
            start++; end--;
        }
    }
}
```

⏱ Time Complexity: **O(n)**

📦 Space Complexity: **O(1)**

@codebuilderhq

6️⃣ Find Missing Number (1 to N)

📌 Problem Statement:

Given an array containing numbers from 1 to n, with one number missing, find that missing number.

- 🔷 **Input:** Array: [1, 2, 4, 5]
- 🔷 **Output:** 3

💻 Java Code:

```
class Main {
    public static void main(String[] args) {
        int[] nums = {1,2,4,5};
        int n = nums.length + 1;
        int totalSum = n * (n + 1) / 2;

        int arraySum = 0;
        for(int num : nums) {
            arraySum += num;
        }

        System.out.println(totalSum - arraySum);
    }
}
```

⏱️ Time Complexity: **O(n)**

📦 Space Complexity: **O(1)**

@codebuilderhq

7️⃣ Reverse Words but Keep Characters Same

📌 Problem Statement:

Reverse the order of words in a sentence, but do not reverse the characters of each word.

- 🔷 **Input:** "I love coding"
- 🔷 **Output:** "coding love I"

💻 Java Code:

```
class Main {
    public static void main(String[] args) {
        String s = "I love coding";
        String[] words = s.split(" ");

        StringBuilder result = new StringBuilder();
        for(int i = words.length - 1; i >= 0; i--) {
            result.append(words[i]);
            if(i != 0) result.append(" ");
        }

        System.out.println(result.toString());
    }
}
```

⏱️ Time Complexity: **O(n)**

📦 Space Complexity: **O(n)**

@codebuilderhq

8️⃣ Palindrome Check (Ignore Case & Special Characters)

📌 Problem Statement:

Check if a string is a palindrome after removing non-alphanumeric characters and ignoring case.

- 🔷 **Input:** "A man, a plan, a canal: Panama"
- 🔷 **Output:** true

💻 Java Code:

```
class Main {
    public static void main(String[] args) {
        String s = "A man, a plan, a canal: Panama";
        s = s.replaceAll("[^a-zA-Z0-9]", "").toLowerCase();

        int left = 0, right = s.length() - 1;
        boolean isPalindrome = true;

        while(left < right) {
            if(s.charAt(left) != s.charAt(right)) {
                isPalindrome = false;
                break;
            }
            left++;
            right--;
        }

        System.out.println(isPalindrome);
    }
}
```

⏱ Time Complexity: **O(n)**

📦 Space Complexity: **O(n)**

@codebuilderhq

9️⃣ Majority Element (More than n/2 times)

📌 Problem Statement:

Find the element that appears more than n/2 times in the array. (Guaranteed to exist)

- 🔷 **Input:** [2, 2, 1, 2, 3, 2, 2]
- 🔷 **Output:** 2

💻 Java Code:

```
class Main {
    public static void main(String[] args) {
        int[] nums = {2,2,1,2,3,2,2};

        int count = 0, candidate = 0;

        for(int num : nums) {
            if(count == 0) candidate = num;
            count += (num == candidate) ? 1 : -1;
        }

        System.out.println(candidate);
    }
}
```

⏱️ Time Complexity: **O(n)**

📦 Space Complexity: **O(1)**

@codebuilderhq